

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I K Sathwik Kumar (192425051) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sathwik Kumar

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q. No	Scenario / Game Question	Platform	Link
1	AI Maze Escape: You are an AI robot trapped in a maze. Move from START to GOAL using the shortest possible path while avoiding walls. Challenge: Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	N-Queens Challenge: Place 12 queens on a 12 × 12 chessboard so that no two queens attack each other. Challenge: Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	Water Jug Puzzle: Two jugs of 11 L and 9 L without markings. Challenge: Measure exactly 8 litres using fill, empty, and pour operations. Play Level 19 and solve it using the minimum number of moves.	Tic-Tac-Toe	https://playtictactoe.org
4	Connect Four AI Challenge: Play against an AI opponent. Challenge: Connect four of your pieces in a row before the AI does by planning your moves carefully.	Connect Four	https://www.mathsisfun.com/games/connect4.html
5	8-Puzzle AI Challenge: Arrange the scrambled 8-puzzle into the correct order by moving one tile at a time. Challenge: Solve the puzzle using the minimum possible number of moves.	8-Puzzle Game	https://8puzzle.org/

Structure of the Report – Game-Based Learning

S. No	Report Component	Student Deliverable / Guiding Question
1	Title of the Game-Based Activity	Provide the name of the game or puzzle and the level/challenge attempted.
2	Game Platform / Link	Mention the platform or website used and provide the corresponding game link.
3	Objective of the Activity	Explain the objective of the game and the target that needs to be achieved.
4	Problem Statement	Describe the problem or challenge given in the game.
5	AI Concept / Domain	Identify the AI concept involved such as State-Space Search, BFS, DFS, A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	Initial State and Goal State	Identify and describe the initial state and the goal state.
7	Actions / Operators Used	List the valid actions or operations available.
8	Solution Strategy	Explain the approach or search strategy used.
9	Step-by-Step Solution	Provide the sequence of moves or actions performed.
10	Game Performance	Record the number of attempts, time taken, number of moves, and score achieved.
11	Successful Completion and Evidence	Provide screenshots or other evidence showing successful completion.
12	Efficiency and Optimality	Analyse whether the solution was optimal or efficient.
13	Analysis and Interpretation	Explain how the selected AI technique helped and interpret the outcome.
14	Critical Thinking and Alternative Solution	Suggest an alternative or improved solution.
15	Learning Outcome / Reflection	Explain what was learned from the activity.
16	Conclusion	Summarize the solution achieved and the significance of AI techniques.
17	References	Provide platform links, books, journals, or other reliable references.

1. AI Maze Escape

Objective of the Activity

The AI Maze Escape game requires an AI robot to travel from the **START** position to the **GOAL** while avoiding walls and blocked paths. The primary objective is to find the shortest possible route using the minimum number of moves. This activity demonstrates how Artificial Intelligence can solve path-finding problems efficiently.

Problem Statement

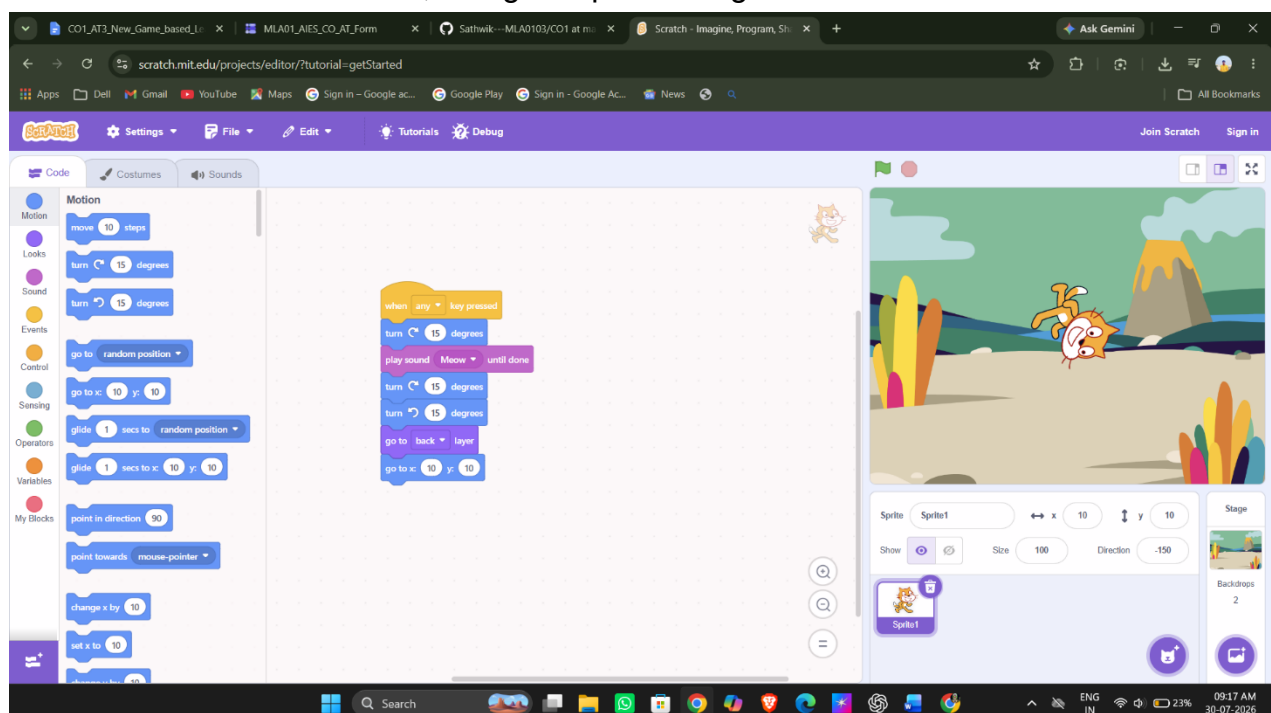
The maze consists of several possible paths, including dead ends and obstacles that prevent direct movement. The robot must carefully explore the available routes and choose the correct path to reach the destination. The challenge is to complete the maze in the least number of steps without crossing any walls.

AI Concept / Solution Strategy

This activity is based on **State Space Search**, where every position inside the maze is treated as a state. The **Breadth First Search (BFS)** algorithm is commonly used because it explores all possible paths level by level and guarantees the shortest path in an unweighted maze. This technique helps the AI identify the optimal solution efficiently.

Key Points

- **Game:** AI Maze Escape
- **Platform:** Scratch
- **Objective:** Reach the GOAL using the shortest path.
- **AI Technique:** Breadth First Search (BFS)
- **AI Domain:** State Space Search
- **Possible Moves:** Up, Down, Left, Right
- **Goal State:** Robot reaches the destination successfully.
- **Real-Life Applications:** Robot navigation, GPS navigation systems, autonomous vehicles, and game pathfinding.



2. N-Queens Challenge

Objective of the Activity

The N-Queens Challenge requires placing **12 queens on a 12 × 12 chessboard** so that no two queens attack each other. The objective is to arrange all queens without conflicts while using the minimum number of moves. This activity demonstrates how AI can solve complex constraint-based problems efficiently.

Problem Statement

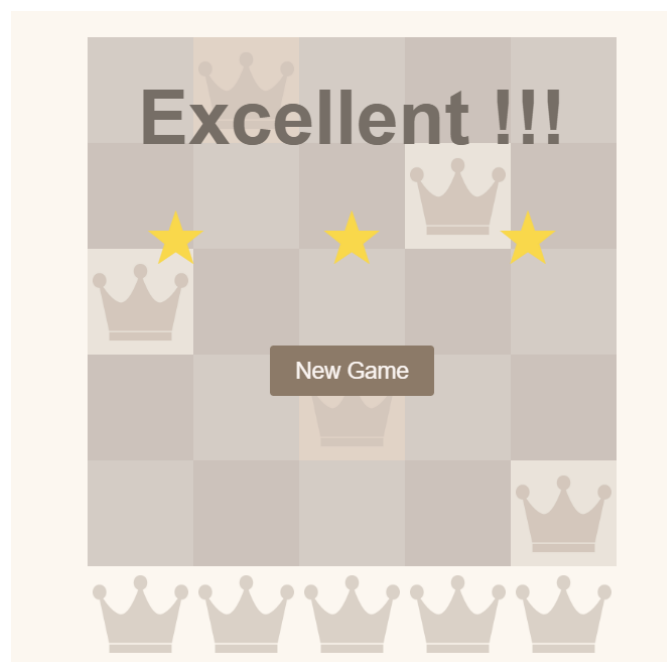
In this puzzle, each queen can move horizontally, vertically, and diagonally. The challenge is to place all 12 queens so that no two queens share the same row, column, or diagonal. The solution must satisfy all constraints simultaneously, making it a classic Artificial Intelligence problem.

AI Concept / Solution Strategy

The N-Queens Challenge is based on the **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. The **Backtracking Search** algorithm is commonly used to place queens one by one while checking for conflicts. If a conflict occurs, the algorithm backtracks and tries another position until a valid arrangement is found.

Key Points

- **Game:** N-Queens Challenge
- **Platform:** Online N-Queens Game
- **Objective:** Place 12 queens without any conflicts.
- **AI Technique:** Backtracking Search
- **AI Domain:** Constraint Satisfaction Problem (CSP)
- **Constraints:** No two queens can attack each other.
- **Goal State:** All 12 queens are placed successfully with zero conflicts.
- **Real-Life Applications:** Timetable scheduling, resource allocation, task scheduling, circuit design, and optimization problems.



Python Code:

```
# 12-Queens Problem using Backtracking
N = 12
board = [[0 for _ in range(N)] for _ in range(N)]
def is_safe(board, row, col):
    # Check left side of the row
    for i in range(col):
        if board[row][i] == 1:
            return False
    # Check upper-left diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    # Check lower-left diagonal
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True
def solve_n_queens(board, col):
    if col >= N:
        return True
    for row in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            if solve_n_queens(board, col + 1):
                return True
            # Backtrack
            board[row][col] = 0
    return False
def print_board(board):
    print("\n12-Queens Solution:\n")
    for row in board:
        for cell in row:
            if cell == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
if solve_n_queens(board, 0):
    print_board(board)
else:
    print("No solution exists.")
```



3)

Water Jug Puzzle

Objective of the Activity

The Water Jug Puzzle challenges the player to measure exactly 8 litres of water using two jugs with capacities of 11 litres and 9 litres. The objective is to achieve the required quantity by using only the allowed operations—filling, emptying, and pouring water between the jugs. This activity helps in understanding how AI solves problems through systematic search techniques.

Problem Statement

The puzzle provides two unmarked water jugs with capacities of 11 litres and 9 litres. Since there are no measurement markings, the player must use a sequence of valid operations to obtain exactly 8 litres in one of the jugs. The challenge is to solve the puzzle using the minimum possible number of moves while following the given rules.

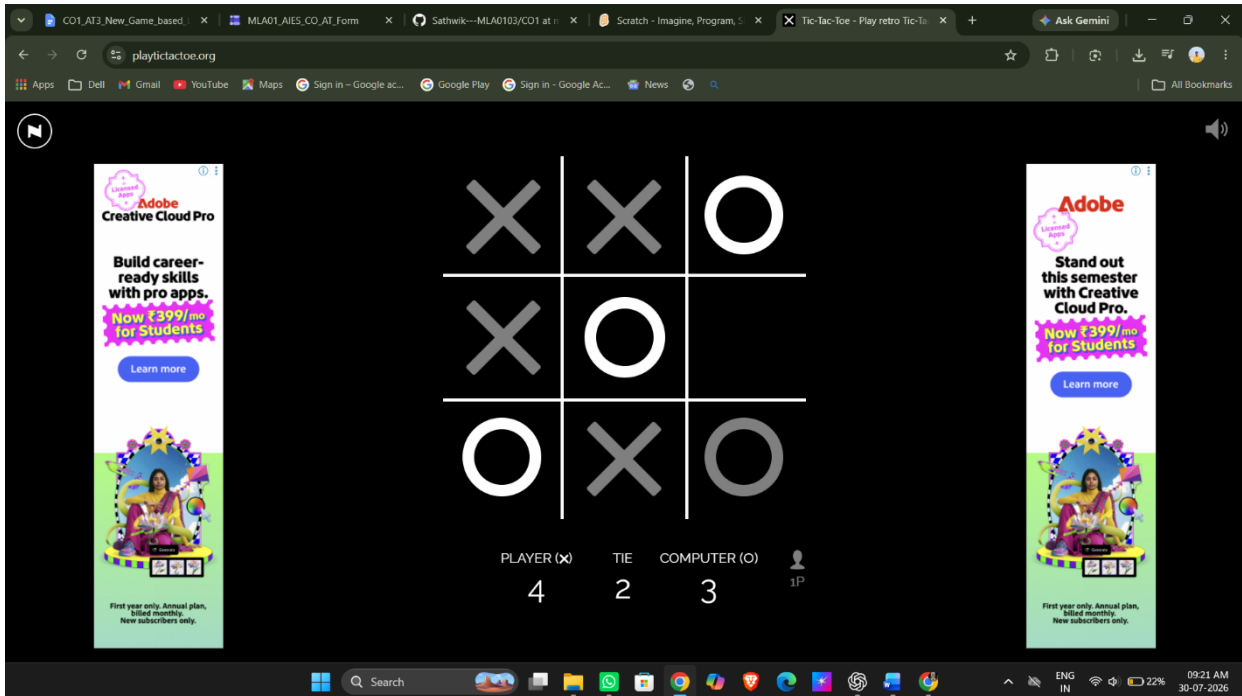
AI Concept / Solution Strategy

The Water Jug Puzzle is a State Space Search problem in Artificial Intelligence. Each combination of water levels in the two jugs represents a different state. The Breadth First Search (BFS) algorithm is commonly used because it explores all possible states systematically and guarantees the shortest sequence of actions to reach the goal state. This makes the solution both efficient and optimal.

Key Points

- Game: Water Jug Puzzle (Level 19)
- Jug Capacities: 11 Litres and 9 Litres
- Objective: Measure exactly 8 litres of water.
- AI Technique: Breadth First Search (BFS)
- AI Domain: State Space Search

- Allowed Operations: Fill, Empty, and Pour
- Goal State: Obtain exactly 8 litres in one of the jugs.
- Real-Life Applications: Resource allocation, process planning, robotics, industrial automation, and decision-making systems.



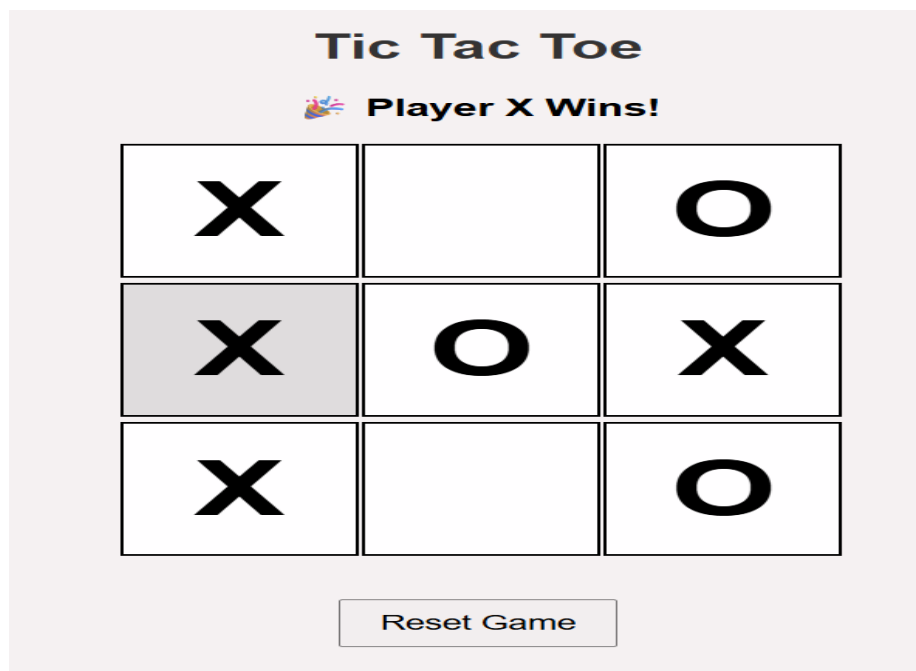
Python Code:

```
# Tic-Tac-Toe Game (Player X vs Player O)
board = [" " for _ in range(9)]
def print_board():
    print()
    print(f" {board[0]} | {board[1]} | {board[2]} ")
    print("----+---+----")
    print(f" {board[3]} | {board[4]} | {board[5]} ")
    print("----+---+----")
    print(f" {board[6]} | {board[7]} | {board[8]} ")
    print()
def check_winner(player):
    win_positions = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for pos in win_positions:
        if all(board[i] == player for i in pos):
            return True
    return False
def board_full():
    return " " not in board
```

```

player = "X"
while True:
    print_board()
    try:
        move = int(input(f"Player {player}, enter position (1-9): ")) - 1
        if move < 0 or move > 8:
            print("Invalid position!")
            continue
        if board[move] != " ":
            print("Position already occupied!")
            continue
        board[move] = player
        if check_winner(player):
            print_board()
            print(f"🎉 Player {player} Wins!")
            break
        if board_full():
            print_board()
            print("👉 Match Draw!")
            break
        player = "O" if player == "X" else "X"
    except ValueError:
        print("Enter numbers only!")

```



4. Connect Four AI Challenge

Objective of the Activity

The Connect Four AI Challenge requires the player to compete against an AI opponent by dropping colored discs into a vertical grid. The objective is to connect **four pieces** in a horizontal, vertical, or diagonal line before the AI achieves the same. This activity demonstrates how Artificial Intelligence makes strategic decisions during gameplay.

Problem Statement

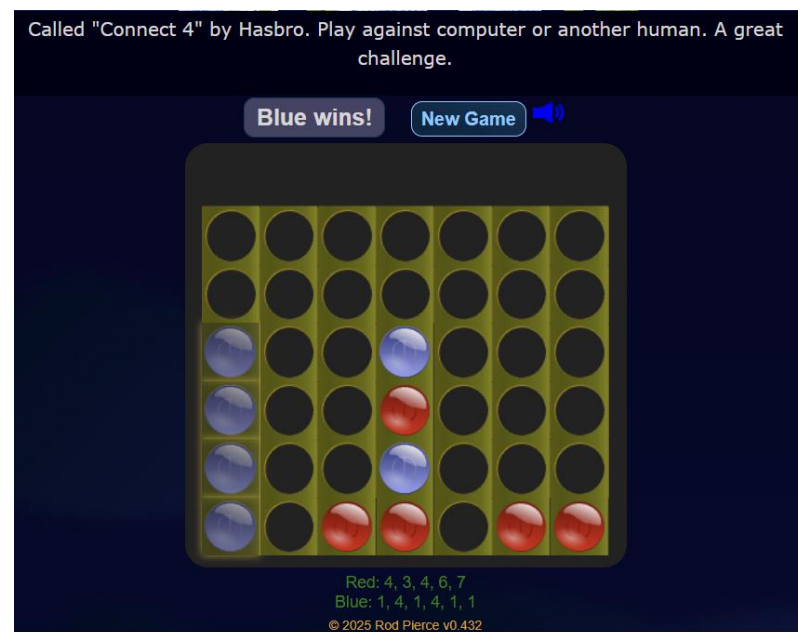
The game is played on a grid where players take turns placing discs into one of the columns. Each disc automatically falls to the lowest available position in that column. The challenge is to carefully plan each move, block the opponent's winning opportunities, and create a sequence of four connected pieces before the AI wins the game.

AI Concept / Solution Strategy

The Connect Four AI Challenge is based on **Game Playing in Artificial Intelligence**. The **Minimax Algorithm** with **Alpha-Beta Pruning** is commonly used to evaluate possible moves and choose the optimal strategy. This technique predicts future game states, maximizes the player's chances of winning, and minimizes the opponent's advantage by selecting the best possible move.

Key Points

- **Game:** Connect Four AI Challenge
- **Platform:** Connect Four
- **Objective:** Connect four pieces before the AI opponent.
- **AI Technique:** Minimax Algorithm with Alpha-Beta Pruning
- **AI Domain:** Game Playing
- **Possible Moves:** Drop a disc into any non-full column.
- **Goal State:** Form four consecutive pieces horizontally, vertically, or diagonally.
- **Real-Life Applications:** Strategic decision-making, game AI, robotics, autonomous planning, and competitive AI systems.



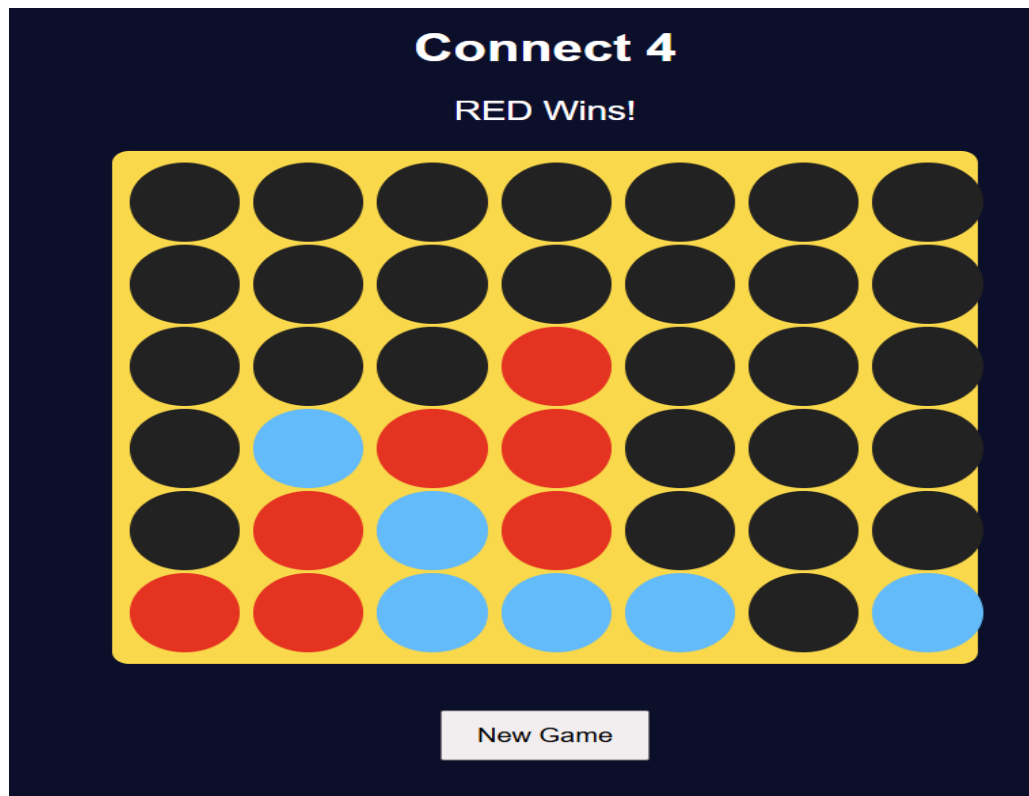
Python Code:

```
import tkinter as tk
from tkinter import messagebox
ROWS = 6
COLS = 7
CELL = 70
board = [["_"] for _ in range(COLS)] for _ in range(ROWS)]
player = "Red"
root = tk.Tk()
root.title("Connect 4")
status = tk.Label(root, text="Red's Turn", font=("Arial", 16))
status.pack()
canvas = tk.Canvas(root, width=COLS * CELL, height=ROWS * CELL, bg="yellow")
canvas.pack()
def draw_board():
    canvas.delete("all")
    for r in range(ROWS):
        for c in range(COLS):
            x1 = c * CELL + 5
            y1 = r * CELL + 5
            x2 = x1 + CELL - 10
            y2 = y1 + CELL - 10
            color = "white"
            if board[r][c] == "Red":
                color = "red"
            elif board[r][c] == "Blue":
                color = "blue"
            canvas.create_oval(x1, y1, x2, y2, fill=color)
def check_win(row, col):
    color = board[row][col]
    directions = [(1,0), (0,1), (1,1), (1,-1)]
    for dr, dc in directions:
        count = 1
        r = row + dr
        c = col + dc
        while 0 <= r < ROWS and 0 <= c < COLS and board[r][c] == color:
            count += 1
            r += dr
            c += dc
        r = row - dr
        c = col - dc
        while 0 <= r < ROWS and 0 <= c < COLS and board[r][c] == color:
            count += 1
            r -= dr
            c -= dc
        if count >= 4:
            return True
    return False
def click(event):
    global player
```

```

col = event.x // CELL
for row in range(ROWS - 1, -1, -1):
    if board[row][col] == "":
        board[row][col] = player
        draw_board()
        if check_win(row, col):
            messagebox.showinfo("Winner", player + " Wins!")
            return
        player = "Blue" if player == "Red" else "Red"
        status.config(text=player + "'s Turn")
        return
canvas.bind("<Button-1>", click)
draw_board()
root.mainloop()

```



5. 8-Puzzle AI Challenge

Objective of the Activity

The 8-Puzzle AI Challenge requires arranging the numbered tiles in the correct order by moving one tile at a time into the empty space. The objective is to transform the scrambled puzzle into the goal configuration using the minimum possible number of moves. This activity helps in understanding how Artificial Intelligence finds optimal solutions to search problems.

Problem Statement

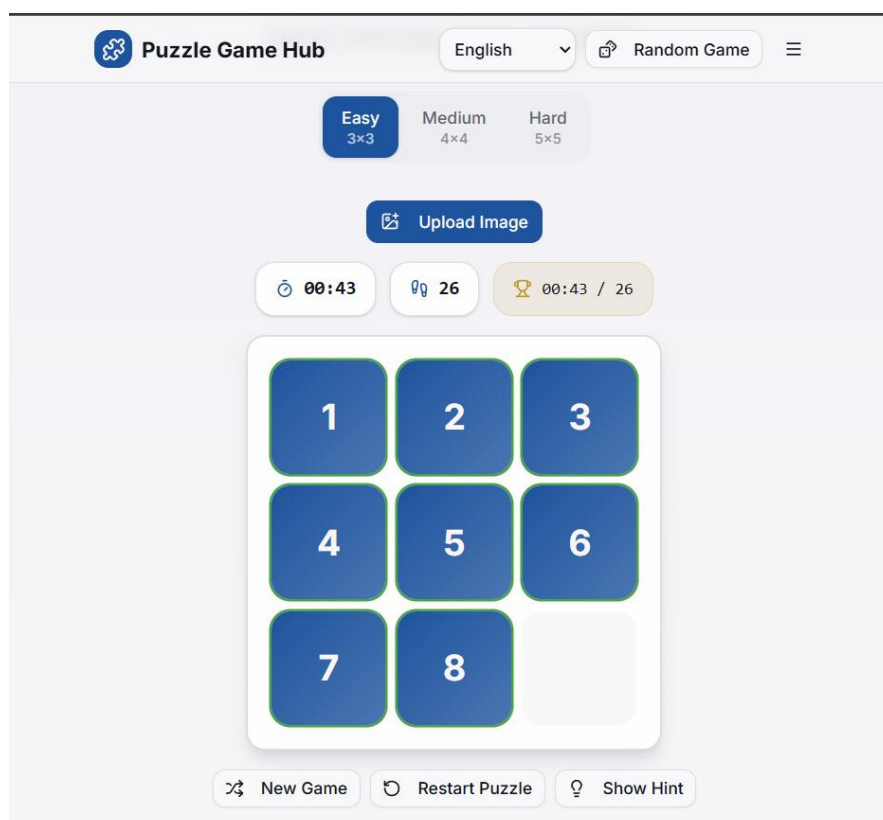
The puzzle consists of **eight numbered tiles** and **one empty space** arranged on a **3 × 3 grid**. The tiles are initially placed in a random order, and only the tile adjacent to the empty space can be moved. The challenge is to arrange the tiles in ascending order while using the fewest possible moves to reach the goal state.

AI Concept / Solution Strategy

The 8-Puzzle Challenge is based on **State Space Search** in Artificial Intelligence. The **A* (A-Star) Search Algorithm** with the **Manhattan Distance Heuristic** is commonly used to solve this puzzle efficiently. The algorithm evaluates possible moves, estimates the remaining distance to the goal, and selects the most optimal path to reach the final arrangement.

Key Points

- **Game:** 8-Puzzle AI Challenge
- **Platform:** 8-Puzzle Game
- **Objective:** Arrange all tiles in the correct order using the minimum number of moves.
- **AI Technique:** A* (A-Star) Search Algorithm
- **AI Domain:** State Space Search
- **Possible Moves:** Move the tile adjacent to the empty space (Up, Down, Left, Right).
- **Goal State:** Tiles arranged in numerical order with the empty space in the final position.
- **Real-Life Applications:** Robot navigation, route planning, automated planning, logistics optimization, and intelligent search systems.



Python Code:

```
import tkinter as tk
import random
from tkinter import messagebox
SIZE = 3
class SlidingPuzzle:
    def __init__(self, root):
        self.root = root
        self.root.title("Sliding Puzzle Game")
        self.tiles = list(range(1, SIZE * SIZE))
        self.tiles.append(0)
        self.shuffle()
        self.buttons = []
        self.moves = 0
        self.label = tk.Label(root, text="Moves: 0", font=("Arial", 14))
        self.label.grid(row=0, column=0, columnspan=SIZE)
        for r in range(SIZE):
            row = []
            for c in range(SIZE):
                btn = tk.Button(root, width=6, height=3,
                               font=("Arial", 20),
                               command=lambda r=r, c=c: self.move(r, c))
                btn.grid(row=r + 1, column=c)
                row.append(btn)
            self.buttons.append(row)
        tk.Button(root, text="New Game", command=self.new_game).grid(
            row=SIZE + 1, column=0, columnspan=SIZE, sticky="we")
        self.update_board()
    def shuffle(self):
        random.shuffle(self.tiles)
    def update_board(self):
        for r in range(SIZE):
            for c in range(SIZE):
                value = self.tiles[r * SIZE + c]
                if value == 0:
                    self.buttons[r][c]["text"] = ""
                    self.buttons[r][c]["bg"] = "lightgray"
                else:
                    self.buttons[r][c]["text"] = str(value)
                    self.buttons[r][c]["bg"] = "steelblue"
                    self.buttons[r][c]["fg"] = "white"
        self.label.config(text=f"Moves: {self.moves}")
    def move(self, r, c):
        index = r * SIZE + c
        empty = self.tiles.index(0)
        er = empty // SIZE
        ec = empty % SIZE
        if abs(r - er) + abs(c - ec) == 1:
            self.tiles[index], self.tiles[empty] = self.tiles[empty], self.tiles[index]
            self.moves += 1
```

```
self.update_board()
if self.tiles == list(range(1, SIZE * SIZE)) + [0]:
    messagebox.showinfo("Congratulations", f"You solved it in {self.moves}
moves!")
def new_game(self):
    self.moves = 0
    self.shuffle()
    self.update_board()
root = tk.Tk()
game = SlidingPuzzle(root)
root.mainloop()
```



Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	Needs Improvement
Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique
Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence
Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking